



به نام خدا
پوشش دیسک واحد
نام درس
نام و نام خانوادگی



دانشگاه صنعتی امیرکبیر، دانشکده مهندسی کامپیوتر
خردادماه 1401

در این گزارش، چهار مورد از الگوریتم‌های تقریبی که اخیراً برای مسئله پوشش دیسک واحد ارائه شده‌اند شرح داده می‌شوند و پس از پیاده‌سازی، با استفاده از چند مجموعه نقطه دنیای واقعی، مورد ارزیابی تجربی قرار می‌گیرند. معیارهای ارزیابی، تعداد دیسک‌های در نظر گرفته شده و زمان اجرای هر الگوریتم خواهد بود. در نهایت عملکرد هر الگوریتم و بهترین الگوریتم‌ها برای هر معیار گزارش می‌شود.

1 شرح مسئله

مجموعه P شامل n نقطه در صفحه داده می‌شود. هدف، پوشش تمام نقاط با استفاده از کمترین تعداد دیسک با شعاع واحد ($r = 1$) است. این مسئله، پوشش دیسک واحد (UDC)¹ نامیده شده و یک مسئله ان‌پی‌سخت² به حساب می‌آید [1]. در کاربردهای مختلف مانند شبکه‌های بی‌سیم، تعیین موقعیت، برنامه‌ریزی حرکت، پردازش تصاویر و... استفاده می‌شود.

Unit Disk Cover¹

NP-hard²

2 مرور الگوریتم‌ها

از سال 1991 تاکنون الگوریتم‌های زیادی ارائه شده‌اند که این مسئله را به صورت تقریبی با فاکتورهای تقریب و پیچیدگی‌های زمانی متفاوت، در ثرم اقلیدسی حل می‌کنند. تاریخچه الگوریتم‌های تقریبی ارائه شده برای این مسئله را نشان می‌دهد.

سال	پیچیدگی زمانی	فاکتور تقریب	مرجع
1991	$O(l^2 n^7)$	$2(1 + \frac{1}{l})$	[2]
1991	$O(n \log S)$	8	[2]
1995	$O(n^3 \log n)$	$O(1)$	[3]
2001	$O(Kn)$	$3(1 + \frac{1}{l})^2$	[4]
2007	$O(n(\log n \log \log n)^2)$	2.8334	[5]
2014	$O(n \log n)$	25/6	[6]
2017	$O(n \log n)$	4	[7]
2018	$O(n \log n)$	4	[8]
2018	—	$O(1.321^d)$	[9]
2019	$O(n)$	7	[10]

Table 1: خلاصه الگوریتم‌های تقریبی ارائه شده برای UDC

در ادامه، به ترتیب الگوریتم‌های ذیل بررسی می‌شوند:

- الگوریتم BLMS که در سال 2017 ارائه شده است [7].
- الگوریتم LL که در سال 2014 ارائه شده است [6].
- الگوریتم DGT که در سال 2018 ارائه شده است [9].
- الگوریتم FastCover که در سال 2019 ارائه شده است [10].

عنوان سه الگوریتم اول، برگرفته از حرف اول نام پژوهشگران مربوط به آن است.

2.1 الگوریتم BLMS

مجموعه P را به عنوان مجموعه نقاط ورودی که در صفحه قرار دارند و C^* را پوشش دیسک بهینه

برای آن در نظر بگیرید. به خاطر داشته باشید که شعاع هر دیسک واحد برابر با 1 است.

تعریف 2.1. در گراف تقاطع دیسک واحد³ $UDIG(P)$ نقاط موجود در مجموعه P رئوس را تشکیل می‌دهند و برای هر جفت $p, q \in P$ یک یال وجود دارد اگر و تنها اگر $|pq| \leq 2$ باشد که $|pq|$ فاصله اقلیدسی بین p و q را نشان می‌دهد.

مشاهده 2.2. برای دو نقطه $p, q \in P$ ، اگر $(p, q) \notin UDIG(P)$ آنگاه p و q نمی‌توانند با یک دیسک واحد پوشش داده شوند.

تعریف 2.3. یک مجموعه مستقل در $UDIG(P)$ ، زیرمجموعه‌ای مانند I از مجموعه P است، به طوری که هیچ یالی بین جفت نقطه‌های موجود در I وجود ندارد. همچنین I مجموعه مستقل حداکثری⁴ نامیده می‌شود، اگر برای هر $p \in P \setminus I$ ، مجموعه $I \cup \{p\}$ در $UDIG(P)$ مستقل نباشد.

فرض کنید I مجموعه مستقل حداکثری در $UDIG(P)$ باشد. طبق اندازه هر مجموعه مستقل در $UDIG(P)$ یک حد پایین⁵ برای تعداد دیسک‌های مورد نیاز به منظور پوشش P است. بنابراین خواهیم داشت:

$$(1) \quad |I| \leq |C^*|$$

مطابق برای پوشش یک دیسک با شعاع 2، هفت دیسک واحد با شعاع 1 لازم و کافی است. بر همین اساس، یک الگوریتم با فاکتور تقریب 7 برای مسئله UDC به دست می‌آید.

فرض کنید I یک مجموعه مستقل حداکثری دلخواه در $UDIG(P)$ باشد. برای هر نقطه $p \in I$ فرض کنید $D(p, 2)$ یک دیسک با مرکزیت نقطه p و شعاع 2 باشد. همچنین در نظر داشته باشید که $d(p)$ دیسک واحدی است که نقطه p را پوشش می‌دهد. علاوه بر این، تمام نقاطی که از مجموعه

Unit Disk Intersection Graph³
Maximal Independent Set⁴
Lower Bound⁵

P توسط $d(p)$ پوشش داده می‌شوند، در $D(p, 2)$ وجود دارند. بنابراین با پوشش $D(p, 2)$ به وسیله 7 دیسک واحد، به ازای همه $p \in I$ ، الگوریتمی با فاکتور تقریب 7 حاصل می‌شود. باید توجه داشت که $UDIG(P)$ ممکن است حداکثر $O(n^2)$ یال داشته باشد. بنابراین پیچیدگی زمانی محاسبه $UDIG(P)$ در بدترین حالت درجه دو خواهد بود.

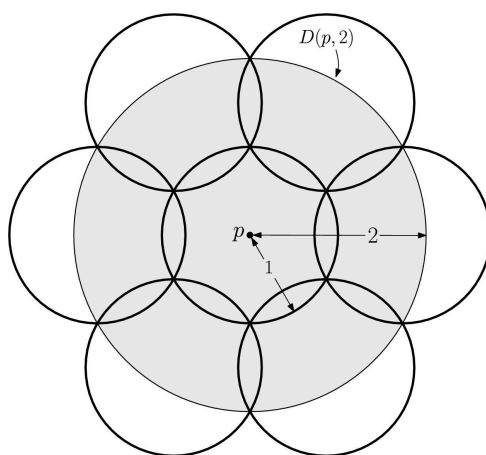


Figure 1: پوشش $D(p, 2)$ با دیسک‌های واحد

در ادامه خواهید دید که چگونه می‌توان فاکتور تقریب را به 4 کاهش داد. p را سمت چپ‌ترین نقطه در مجموعه P در نظر بگیرید. در موارد خاص که مقادیر x نقاط با هم برابر است، برای انتخاب سمت چپ‌ترین نقطه، کمتر بودن مقدار y را ملاک قرار می‌دهیم. فرض کنید خط عمودی l از نقطه p عبور کند؛ $R(p)$ اشتراک $D(p, 2)$ با نیم صفحه⁶ سمت راست l خواهد بود؛ یعنی $R(p)$ نیم دیسک سمت راست $D(p, 2)$ است (مطابق).

همان‌طور که قبلاً هم توضیح داده شد، تمام نقاط مجموعه P که با $d(p)$ پوشش داده شده‌اند، در $D(p, 2)$ و به تبع آن در $R(p)$ نیز قرار دارند. مطابق، نیم دیسک $R(p)$ می‌تواند با 4 دیسک واحد پوشش داده شود. قسمت دوم شکل، حالتی از موقعیت قرار گرفتن 7 نقطه را نشان می‌دهد که برای پوشش آنها حداقل به 4 دیسک واحد نیاز است.

⁶Half-plane

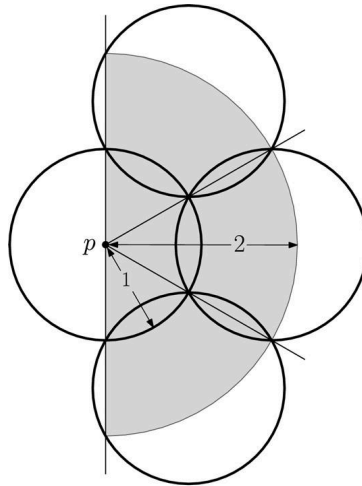


Figure 2: پوشش $R(p)$ با دیسک‌های واحد

برای نقطه‌ای مانند p و مجموعه نقاط I فاصله $d(p, I)$ برابر است با کمترین فاصله اقلیدسی بین p و هر نقطه موجود در I . اگر مجموعه I تهی باشد، فاصله بینهایت در نظر گرفته می‌شود. الگوریتم پیشنهادی با فاکتور تقریب 4 را نشان می‌دهد. خروجی آن، مجموعه‌ای از دیسک‌های واحد با نام C است که مجموعه نقاط P را پوشش می‌دهند. ابتدا لیستی از نقاط که از چپ به راست مرتب شده‌اند ایجاد می‌شود. سپس هردفعه اولین عنصر p از لیست انتخاب و حذف می‌شود. اگر فاصله $d(p, I) \leq 2$ باشد، نشان‌دهنده این است که قبلاً نقطه p توسط یکی از دیسک‌های مجموعه C پوشش داده شده است. در غیر این صورت، نیم‌دیسک $R(p)$ با 4 دیسک واحد پوشش داده می‌شود و به مجموعه C اضافه می‌شود. در نهایت مجموعه دیسک‌های واحد C به‌عنوان خروجی حاصل می‌شود.

```

1:  $C = \emptyset$ 
2:  $I = \emptyset$ 
3:  $L =$  List of points in  $P$  sorted from left to right
4: while  $L$  is not empty do
5:      $p =$  first element of  $L$ 
6:     if  $d(p, I) > 2$  then
7:         Cover  $R(p)$  by 4 unit disks  $c_1, c_2, c_3, c_4$ 
8:          $C = C \cup \{c_1, c_2, c_3, c_4\}$ 
9:          $I = I \cup \{p\}$ 
10:    end if
11:     $L = L - \{p\}$ 
12: end while
13: return  $C$ 

```

الگوریتم 1: نسخه اولیه BLMS

در هر تکرار، نقطه p به مجموعه I اضافه می‌شود، اگر و تنها اگر $d(p, I) > 2$ باشد. بنابراین p در $UDIG(P)$ به هیچ نقطه‌ای از I متصل نیست. همچنین بعد از خاتمه الگوریتم، I یک مجموعه مستقل حداکثری خواهد بود.

قضیه 2.4. فاکتور تقریب برای مسئله پوشش دیسک واحد، 4 است.

برهان. مجموعه نقاط I و مجموعه دیسک‌های واحد C را در پایان الگوریتم، در نظر بگیرید.

بر اساس رابطه برابری Equation 1 می‌دانیم که نامساوی $|I| \leq |C^*|$ برقرار است. به ازای نقاط

$p \in I$ هر نقطه $q \in P$ در یک نیم‌دیسک $R(p)$ قرار دارد (ممکن است $q = p$ باشد). چون برای

هر نقطه $p \in I$ ، نیم‌دیسک $R(p)$ با 4 دیسک واحد پوشش داده می‌شود، مجموعه C مجموعه P را پوشش می‌دهد. بنابراین رابطه $|C| \leq 4|I| \leq 4|C^*|$ برقرار است.

پیچیدگی زمانی $O(n \log n + n \cdot t(d))$ است. $t(d)$ پیچیدگی زمانی محاسبه فاصله $d(p, I)$ را نشان می‌دهد. محاسبه این فاصله با پیچیدگی زمانی $O(\log^2 n)$ قابل انجام است [11]. بنابراین پیچیدگی نهایی الگوریتم، $O(\log^2 n)$ است که در ادامه با استفاده از تکنیک جاروی صفحه، بهبود می‌یابد. به جای محاسبه فاصله $d(p, I)$ کافی است بدانیم فاصله نقطه p از مجموعه I از 2 بیشتر است یا نه؛ یعنی به یک مسئله تصمیم‌گیری تبدیل شود. به تدریج که یک نقطه جدید مانند p به مجموعه I اضافه می‌شود، نقاط مجموعه P که در نیم‌دیسک $R(p)$ قرار دارند، حذف می‌شوند. بر همین اساس، یک الگوریتم با استفاده از تکنیک جاروی صفحه و پیچیدگی زمانی $O(n \log n)$ ارائه می‌شود.

در، ابتدا نقاط بر اساس مؤلفه x از چپ به راست مرتب شده و در صف رخدادها درج می‌شوند. خط جارو به صورت عمودی از چپ به راست بر روی نقاط حرکت می‌کند. به هر نقطه که می‌رسد، اگر نقطه انتهایی یک نیم‌دیسک باشد، نقطه ابتدایی مربوط به آن نیم‌دیسک را از درخت وضعیت حذف می‌کند. در غیر این صورت، دو نقطه همسایه از بالا و دو نقطه همسایه از پایین را در درخت وضعیت برای این نقطه پیدا می‌کند. اگر فاصله آن با حداقل یکی از این 4 همسایه کمتر یا مساوی 2 باشد، به این معنا است که نقطه مورد نظر قبلاً در یک نیم‌دیسک قرار گرفته و تحت پوشش است و نیاز به اقدام خاصی نیست. در غیر این صورت، یک نیم‌دیسک به شعاع 2 و مرکز آن نقطه در نظر گرفته می‌شود و مرکز 4 دیسک واحد پوشش‌دهنده آن نیم‌دیسک محاسبه و به عنوان جواب گزارش می‌شود. سپس آن نقطه به عنوان نقطه ابتدایی نیم‌دیسک، در مکان مناسب خود در درخت وضعیت درج می‌شود. همچنین، نقطه انتهایی این نیم‌دیسک با اضافه کردن 2 واحد به مؤلفه x مرکز آن، محاسبه شده و در جای مناسب در صف رخدادها درج می‌شود. لازم به ذکر است که

درخت وضعیت، شامل نقاط ابتدایی نیم‌دیسک‌هایی است که در هر لحظه با خط جارو متقاطع‌اند و به صورت مرتب شده بر اساس مؤلفه y از پایین به بالا قرار گرفته‌اند.

```
1: Initialize an empty event queue Q. Insert the points in ascending
   order of their x-coordinates into Q.
2: Initialize an empty BST status structure T.
3: Initialize an empty list C.
4: while Q is not empty do
5:     Determine the next event point p in Q and delete it.
6:     if p is an end-point then
7:         Delete the start-point of the corresponding half-disk from T.
8:     else
9:         Find the 2 top and the 2 bottom neighbors of p in T.
10:        if The distance between p and all of these 4 neighbors is
           greater than 2 then
11:            Calculate center points of the 4 unit disks which cover
              half-disk of point p and insert them into C.
12:            Insert p into T.
13:            Insert the end-point  $q=(p_x + 2, p_y)$  into Q.
14:        end if
15:    end if
16: end while
17: return C
```

الگوریتم 2: نسخه بهبود یافته BLMS با تکنیک جاروی صفحه

2.2 الگوریتم LL

در این الگوریتم، صفحه به نوارهای عمودی با عرض $\sqrt{3}$ تقسیم می‌شود. از هر نوار، یک جواب تقریبی با مرتب‌سازی نقاط بر اساس مؤلفه y به صورت نزولی به دست می‌آید. نقطه بعدی درون یک نوار که هنوز پوشش داده نشده است، با قرار دادن یک دیسک در پایین‌ترین مکان ممکن، پوشش داده می‌شود. مرکز این دیسک‌ها، روی خطوط عمودی که نوارها را به دو قسمت تقسیم

می‌کنند قرار می‌گیرد. جواب نهایی با اجتماع جواب همه نوارها حاصل می‌شود. این سامانه نواری، 5 مرتبه به سمت راست و هر دفعه به اندازه $\sqrt{3}/6$ شیفت داده می‌شود. در هر شیفت، یک جواب به دست می‌آید. از بین این 6 جواب، آن جوابی که کمترین دیسک را استفاده کرده باشد به عنوان جواب نهایی در نظر گرفته می‌شود. جزئیات بیشتر در موجود است. فاکتور تقریب این الگوریتم $4.17 \approx 25/6$ است.

```

1: Disk-Centers  $\leftarrow \emptyset$ , min  $\leftarrow n+1$ 
2: Sort P w.r.t x-coordinate in  $O(n \log n)$  time
3: for  $i \in \{0,1,2,3,4,5\}$  do
4:   current  $\leftarrow 1$ , C  $\leftarrow \emptyset$ , right  $\leftarrow P[1]_x + (i\sqrt{3})/6$ 
5:   while current  $\leq n$  do
6:     index  $\leftarrow$  current
7:     while  $P[\text{current}]_x < \text{right}$  and current  $\leq n$  do
8:       current  $\leftarrow$  current + 1
9:     end while
10:    x-of-restriction-line  $\leftarrow$  right -  $\sqrt{3}/2$ , segments  $\leftarrow \emptyset$ 
11:    for j  $\leftarrow$  index to current-1 do
12:      d  $\leftarrow P[j]_x - \text{x-of-restriction-line}$ , y  $\leftarrow \sqrt{1-d^2}$ 
13:      Create a segment s having the endpoints
        (x-of-restriction-line,  $P[j]_y+y$ ) and
        (x-of-restriction-line,  $P[j]_y-y$ ) and insert it
        into segments
14:    end for
15:    Sort segments in non-ascending order based on y-coordinates
        of their tops. Greedily stab them by choosing the stabbing
        point as low as possible, while still stabbing the topmost
        unstabbed segment. Put the stabbing points (the disk centers)
        in C
16:    Increment right by a multiple of  $\sqrt{3}$  such that
         $P[\text{current}]_x - \text{right} \leq \sqrt{3}$ 
17:  end while
18:  if  $|C| < \text{min}$  then
19:    Disk-Centers  $\leftarrow C$ , min  $\leftarrow |C|$ 
20:  end if
21: end for
22: return Disk-Centers

```

الگوریتم 3: محاسبه موقعیت دیسک‌های واحد با استفاده از LL

2.3 الگوریتم DGT

این الگوریتم، ساده و برخط است. به ازای هر نقطه‌ای که تاکنون تحت پوشش قرار نگرفته است، یک دیسک واحد به مرکز آن نقطه ایجاد می‌شود. فاکتور تقریب آن در صفحه، 5 است. در فضای با ابعاد d دارای فاکتور تقریب $O(1.321^d)$ است. برای مشاهده توصیف سطح بالا، به مراجعه نمایید.

```

1: Disk-Centers  $\leftarrow \emptyset$ 
2: for  $p \in P$  do
3:   if the distance from  $p$  to the nearest point in Disk-Centers is  $>1$  then
4:     Disk-Centers  $\leftarrow$  Disk-Centers  $\cup \{p\}$ 
5:   end if
6: end for
7: return Disk-Centers

```

الگوریتم 4: محاسبه موقعیت دیسک‌های واحد با استفاده از DGT

2.4 الگوریتم FastCover

در این الگوریتم، از یک شبکه⁷ با مربع‌های به ضلع $\sqrt{2}$ استفاده می‌شود. هر مربع از این شبکه، می‌تواند توسط یک دیسک به شعاع واحد محاط شود. به ازای هر نقطه، اگر توسط یکی از دیسک‌هایی که قبلاً قرار گرفته است تحت پوشش باشد، عملی انجام نمی‌شود؛ در غیر این صورت، یک دیسک واحد به مرکز مربعی که آن نقطه درونش قرار گرفته است، ایجاد می‌شود.

در پیاده‌سازی برای جستجوی سریع‌تر، از یک جدول درهم‌ساز⁸ استفاده می‌شود. در این جدول، مختصات مرکز دیسک‌هایی که اضافه شده‌اند ذخیره می‌شود. برای جلوگیری از مشکلات اعداد اعشاری، از یک جفت عدد صحیح برای نمایش مرکز هر دیسک استفاده می‌شود. مختصات حقیقی می‌تواند با ضرب کردن هر عدد صحیح در $\sqrt{2}$ و اضافه کردن $1/\sqrt{2} = \sqrt{2}/2$ به آن به دست بیاید. برای به دست آوردن اعداد صحیح از روی یک نقطه، مؤلفه‌های x, y آن بر $\sqrt{2}$ تقسیم می‌شود.

Grid⁷
Hash table⁸

درواقع این اعداد صحیح، مربع مربوط به آن نقطه را در شبکه نشان می‌دهد. این فرآیند در قابل ملاحظه است.

این الگوریتم دارای فاکتور تقریب 7 و پیچیدگی زمانی $O(n)$ است. یک الگوریتم برخط به حساب می‌آید و هیچ پیش‌پردازشی مثل مرتب‌سازی روی نقاط انجام نمی‌دهد. به‌اندازه $O(s)$ حافظه اضافی مصرف می‌کند که s نشان‌دهنده اندازه پوشش تولید شده است.

```
1:  $H \leftarrow \emptyset$ ; Disk-Centers  $\leftarrow \emptyset$ 
2: for  $p \in P$  do
3:    $i \leftarrow \lfloor p_x/\sqrt{2} \rfloor$ ;  $j \leftarrow \lfloor p_y/\sqrt{2} \rfloor$ 
4:   if  $(i,j) \notin H$  then
5:     insert  $(i,j)$  into  $H$  and  $(\sqrt{2} \cdot i + 1/\sqrt{2}, \sqrt{2} \cdot j + 1/\sqrt{2})$ 
        into Disk-Centers
6:   end if
7: end for
8: return Disk-Centers
```

الگوریتم 5: محاسبه موقعیت دیسک‌های واحد با استفاده از FastCover

این الگوریتم را می‌توان کمی بهبود داد. وقتی که یک نقطه در یک مربع از شبکه قرار می‌گیرد، ممکن است توسط 4 دیسک واحد که مربوط به مربع‌های همسایه است و قبلاً اضافه شده‌اند تحت پوشش باشد (مطابق). بنابراین بهتر است برای کاهش تعداد دیسک‌ها این شرایط نیز بررسی شود. در جزئیات نسخه بهبودیافته ذکر شده است.

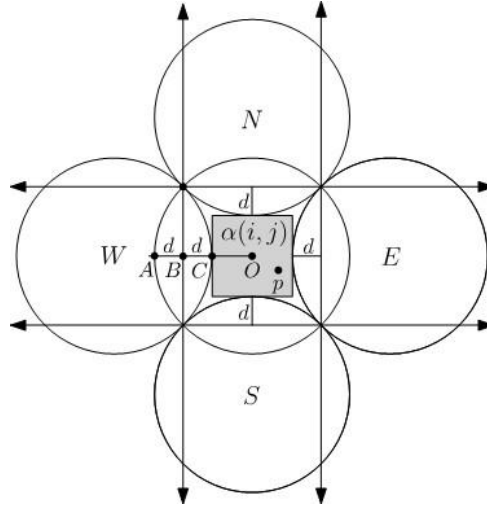


Figure 3: امکان پوشش یک نقطه با دیسک‌های مربع‌های همسایه

```

1:  $H \leftarrow \emptyset$ ; Disk-Centers  $\leftarrow \emptyset$ 
2: for  $p \in P$  do
3:    $i \leftarrow \lfloor p_x / \sqrt{2} \rfloor$ ;  $j \leftarrow \lfloor p_y / \sqrt{2} \rfloor$ 
4:   if  $(i, j) \in H$  then
5:     update  $B(i, j)$  using  $p$  //  $p$  is already covered by  $D(i, j)$ 
6:   else if  $p_x \geq \sqrt{2}(i+1.5) - 1$  and  $(i+1, j) \in H$  and
       distance( $p$ ,  $(\sqrt{2}(i+1)+1/\sqrt{2}, \sqrt{2} \cdot j+1/\sqrt{2})$ )  $\leq 1$  then
7:     continue //  $p$  is covered by the grid-disk  $E$  placed before
8:   else if  $p_x \leq \sqrt{2}(i-0.5) + 1$  and  $(i-1, j) \in H$  and
       distance( $p$ ,  $(\sqrt{2}(i-1)+1/\sqrt{2}, \sqrt{2} \cdot j+1/\sqrt{2})$ )  $\leq 1$  then
9:     continue //  $p$  is covered by the grid-disk  $W$  placed before
10:  else if  $p_y \geq \sqrt{2}(j+1.5) - 1$  and  $(i, j+1) \in H$  and
       distance( $p$ ,  $(\sqrt{2} \cdot i+1/\sqrt{2}, \sqrt{2}(j+1)+1/\sqrt{2})$ )  $\leq 1$  then
11:    continue //  $p$  is covered by the grid-disk  $N$  placed before
12:  else if  $p_y \leq \sqrt{2}(j-0.5) + 1$  and  $(i, j-1) \in H$  and
       distance( $p$ ,  $(\sqrt{2} \cdot i+1/\sqrt{2}, \sqrt{2}(j-1)+1/\sqrt{2})$ )  $\leq 1$  then
13:    continue //  $p$  is covered by the grid-disk  $S$  placed before
14:  else
15:    insert  $(i, j)$  into  $H$  and  $(\sqrt{2} \cdot i + 1/\sqrt{2}, \sqrt{2} \cdot j + 1/\sqrt{2})$ 
        into Disk-Centers
16:  end if
17: end for
18: return Disk-Centers

```

الگوریتم 6: محاسبه موقعیت دیسک‌های واحد با استفاده از FastCover+

بهبود دیگری می‌توان روی الگوریتم اعمال نمود. اگر نقاط موجود در دو دیسک مجاور را بتوان با یک دیسک پوشش داد، یعنی قطر مجموعه نقاط هر دو دیسک کمتر از 2 باشد، می‌توان آن دو دیسک را ادغام نمود. برای ادغام، یک دیسک واحد به مرکز وسط قطر مجموعه نقاط در نظر گرفته می‌شود. برای اینکه محاسبه قطر، تأثیری روی پیچیدگی زمانی الگوریتم نداشته باشد، به همراه هر دیسک واحدی که ایجاد می‌شود، یک مستطیل مرزی نیز برای مجموعه نقاط موجود در آن نگهداری می‌شود. هر دفعه که یک نقطه جدید تحت پوشش یک دیسک واحد قرار می‌گیرد، ابعاد مستطیل مرزی مربوط به آن نیز بروزرسانی می‌شود. این بروزرسانی در $O(1)$ قابل انجام است. جزئیات بیشتر در ذکر شده است.

```

1:  $H \leftarrow \emptyset$ ; Disk-Centers  $\leftarrow \emptyset$ 
2: for  $p \in P$  do
3:    $i \leftarrow \lfloor p_x/\sqrt{2} \rfloor$ ;  $j \leftarrow \lfloor p_y/\sqrt{2} \rfloor$ 
4:   if  $(i,j) \in H$  then
5:     update  $B(i,j)$  using  $p$  //  $p$  is already covered by  $D(i,j)$ 
6:   else if  $p_x \geq \sqrt{2}(i+1.5) - 1$  and  $(i+1,j) \in H$  and
       distance( $p$ ,  $(\sqrt{2}(i+1)+1/\sqrt{2}, \sqrt{2}j+1/\sqrt{2})$ )  $\leq 1$  then
7:     update  $B(i+1,j)$  using  $p$  //  $p$  is covered by grid-disk E
8:   else if  $p_x \leq \sqrt{2}(i-0.5) + 1$  and  $(i-1,j) \in H$  and
       distance( $p$ ,  $(\sqrt{2}(i-1)+1/\sqrt{2}, \sqrt{2}j+1/\sqrt{2})$ )  $\leq 1$  then
9:     update  $B(i-1,j)$  using  $p$  //  $p$  is covered by grid-disk W
10:  else if  $p_y \geq \sqrt{2}(j+1.5) - 1$  and  $(i,j+1) \in H$  and
       distance( $p$ ,  $(\sqrt{2}i+1/\sqrt{2}, \sqrt{2}(j+1)+1/\sqrt{2})$ )  $\leq 1$  then
11:    update  $B(i,j+1)$  using  $p$  //  $p$  is covered by grid-disk N
12:  else if  $p_y \leq \sqrt{2}(j-0.5) + 1$  and  $(i,j-1) \in H$  and
       distance( $p$ ,  $(\sqrt{2}i+1/\sqrt{2}, \sqrt{2}(j-1)+1/\sqrt{2})$ )  $\leq 1$  then
13:    update  $B(i,j-1)$  using  $p$  //  $p$  is covered by grid-disk S
14:  else
15:    insert  $(i,j)$  into  $H$  and initialize  $B(i,j)$  using  $p$ 
16:  end if
17: end for
18: while there is a grid-disk  $(i,j) \in H$  that is not considered yet do
19:   if there is a grid disk  $(k,\ell) \in H$  such that  $|i-k| \leq 1$ ,  $|j-\ell| \leq 1$ 
       and the diagonal-length of the bounding-box
        $B := B(i,j) \cup B(k,\ell)$  is at most 2 then
20:     remove  $(i,j)$  and  $(k,\ell)$  from  $H$  and add the center of  $B$  to
       Disk-Centers
21:   end if
22: end while
23: for every grid-disk  $(i,j) \in H$  do
24:   insert  $(\sqrt{2}i + 1/\sqrt{2}, \sqrt{2}j + 1/\sqrt{2})$  into Disk-Centers
25: end for
26: return Disk-Centers

```

الگوریتم 7: محاسبه موقعیت دیسک‌های واحد با استفاده از ++FastCover

Table 2: نتایج ارزیابی الگوریتم‌ها

4 نتیجه‌گیری

در مجموعه نقطه‌های دنیای واقعی، اگر معیار با اهمیت‌تر برای ارزیابی، تعداد دیسک‌های استفاده شده باشد، الگوریتم LL عملکرد بهتری دارد. هرچند که الگوریتم FastCover⁺⁺ در برخی موارد، هم از لحاظ تعداد دیسک‌ها و هم زمان اجرا، عملکرد بهتری داشته است.

اگر معیار مهم‌تر، زمان اجرا باشد الگوریتم FastCover عملکرد بهتری داشته است؛ اما چون تفاوت چندانی با نسخه بهبودیافته خود که تعداد دیسک‌های کمتری تولید می‌کند ندارد، استفاده از FastCover⁺⁺ پیشنهاد می‌شود.

اگر هر دو معیار تعداد دیسک‌ها و زمان اجرا با اهمیت باشد، استفاده از الگوریتم FastCover⁺⁺ توصیه می‌شود؛ چرا که تعادل خوبی بین هر دو معیار برقرار می‌کند [12].

Bibliography 4

- [1] S. L. Tanimoto, “Optimal packing and covering in the plane و R. J. Fowler, M. S. Paterson
Information processing letters, ج 12, ش 3, صص 137–133, 1981.
- [2] T. F. Gonzalez, “Covering a set of points in multidimensional space,” *Information
processing letters*, ج 40, ش 4, صص 188–181, 1991.
- [3] M. T. Goodrich, “Almost optimal set covers in finite VC-dimension,” و H. Brönnimann
Discrete & Computational Geometry, ج 14, ش 4, صص 479–463, 1995.
- [4] J. Bruck, “A geometric theorem for approximate disk و M. Franceschetti, M. Cook
covering algorithms,” 2001.
- [5] M. Abdelguerfi, “An almost linear time 2.8334-approximation algorithm و B. Fu, Z. Chen
”for the disc covering problem *در International Conference on Algorithmic Applications*
in Management, 2007, صص 326–317.
- [6] D. Lu, “A fast 25/6-approximation for the minimum unit disk cover problem,” و P. Liu
arXiv, 2014.
- [7] M. Smid, “Approximation algorithms for the unit disk و A. Biniaz, P. Liu, A. Maheshwari
Computational Geometry, ج 60, صص 18–8, 2017.
- [8] S. N. Hashemi, “A simple greedy approximation algorithm for the unit و M. Imanparast
disk cover problem,” *AUT Journal of Mathematics and Computing*, ج 1, ش 1, صص 47–
55, 2020.
- [9] C. D. Tóth, “Online unit covering in Euclidean space,” و A. Dumitrescu, A. Ghosh
Theoretical Computer Science, ج 809, صص 230–218, 2020.

- [10] R. Shevchenko, “Unit disk cover for massive point sets” و A. Ghosh, B. Hicks [10] در
International Symposium on Experimental Algorithms, 2019, صص 142–157.
- [11] J. B. Saxe, “Decomposable searching problems I. Static-to-dynamic” و J. L. Bentley [11]
Journal of Algorithms, transformation,” ج 1, ش 4, صص 301–358, 1980.
- [12] R. Shevchenko, “Experiments with Unit” و R. Friederich, M. Graham, A. Ghosh, B. Hicks [12]
Disk Cover Algorithms for Covering Massive Pointsets, *arXiv*, 2022.

4 پیوست

همان‌طور که در ملاحظه شد، بهترین الگوریتم‌هایی که تاکنون برای مسئله UDC ارائه شده‌اند، دارای فاکتور تقریب 4 هستند. اینجانب، توجه زیادی به این مسئله با هدف بهبود فاکتور تقریب و نگارش مقاله نمودم. ایده‌های مختلفی برای بهبود فاکتور تقریب به ذهنم رسید که پس از بررسی‌های فراوان متوجه شدم برخی از آنها اشتباه است و برخی دیگر را به دلیل کمبود زمان نتوانستم به‌طور دقیق بررسی کنم. در ادامه تعدادی از آنها را بیان می‌کنم.

ایده اول، با هدف کاهش فاکتور تقریب از 4 به 3 بود. در نشان داده شد که برای پوشش یک نیم‌دیسک به شعاع 2، دقیقاً به 4 دیسک واحد احتیاج است. بر اساس این شکل، برای پوشش یک ربع‌دیسک نیز دقیقاً به 3 دیسک واحد احتیاج است. با فرض اینکه اشتراک دو نیم‌دیسک به شعاع 2 حداکثر با یک ربع‌دیسک قابل پوشش است، می‌توان یک‌بار مطابق خط جارو را از چپ به راست و بار دیگر از بالا به پایین حرکت داد. طی این دو مرحله، تعدادی نیم‌دیسک حاصل می‌شود. نواحی از نیم‌دیسک‌ها که با یکدیگر هم‌پوشانی پیدا می‌کنند (اشتراک نیم‌دیسک‌ها) مواردی است که لازم است با دیسک‌های واحد پوشش داده شود. پس از بررسی مشخص شد که این ایده عملی نیست. چون فرض اولیه اشتباه است و حالت‌هایی وجود دارد که اشتراک دو نیم‌دیسک بیشتر از حد تصور می‌شود و نمی‌توان آن را با یک ربع‌دیسک پوشش داد (مطابق).

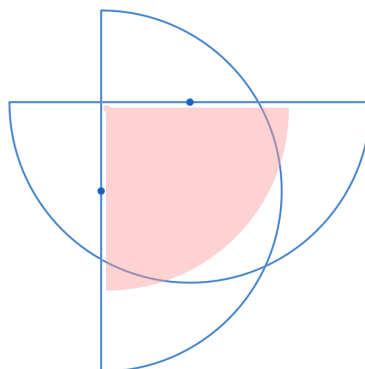


Figure 4: عدم امکان پوشش اشتراک دو نیم‌دیسک با ربع‌دیسک

ایده دوم نیز با هدف کاهش فاکتور تقریب از 4 به 3 بود. به جای نیم‌دیسک‌های با شعاع 2، ربع‌دیسک‌هایی به شعاع 2 در نظر گرفته می‌شود که هرکدام با 3 دیسک واحد قابل پوشش هستند. در جزئیات ایده بیان شده است. در این الگوریتم، خط جارو از بالا به پایین بر روی نقاط حرکت می‌کند. پس از بررسی، مشخص شد که این ایده نیز، عملی نیست. زیرا مانند حالت‌هایی وجود دارد که ربع‌دیسک‌های زیادی با یکدیگر همپوشانی پیدا می‌کنند و در نهایت فاکتور تقریب، $O(n)$ می‌شود.

```

1: Initialize an empty event queue Q. Insert the points in descending
   order of their y-coordinates into Q. If two points have the same
   y-coordinate, the one with smaller x-coordinate has higher priority.
2: Initialize an empty BST status structure T.
3: Initialize an empty list C.
4: while Q is not empty do
5:     Determine the next event point p in Q and delete it.
6:     if p is an end-point then
7:         Delete the start-point of the corresponding quarter disk from T.
8:     else
9:         Find the two left neighbors p', p'' of p in T. If p has the
           same x-coordinate with a point in T, consider the point with the
           higher y-coordinate as the left.
10:        if |pp'| > 2 and |pp''| > 2 then
11:            Calculate center points of the 3 unit disks which cover
              quarter disk of point p and insert them into C.
12:            Insert p into T.
13:            Insert the end-point q=(px, py + 2) into Q.
14:        end if
15:    end if
16: end while
17: return C

```

الگوریتم 8: ایده دوم برای کاهش فاکتور تقریب به 3

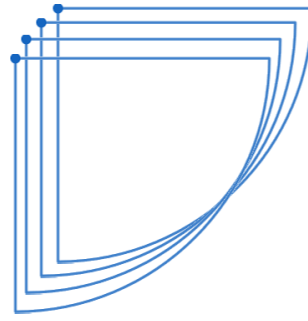


Figure 5: حالتی که ربع‌دیسک‌های زیادی همپوشانی پیدا می‌کنند